Cloud Economics: Cloud Specs (A WebApp with RAG-Powered Chatbot)

Nitish Devrani
University of Technology Nuremberg (UTN)
Nuremberg, Germany
nitish.devrani@utn.de

***Abstract*—This report presents a web-based cloud advisory platform that helps users find cost-effective compute instances across AWS, Azure, and GCP. The platform offers three main features: a multi-cloud recommendation system that lets users filter and compare instances across all three providers, a RAG-powered chatbot that answers cloud-pricing questions using indexed real-world data, and a conversational shopping agent that walks users through building a complete infrastructure bundle. The RAG pipeline runs on the Arcana vector-database service with `qwen3-embedding-4b` embeddings over 39.55 MB of cloud-pricing data (3 947 chunks, 107 MB indexed). A dual-prompt strategy rewrites vague user queries before retrieval to get better results. The React frontend is served alongside Meta-Llama-3.1-8B-Instruct and Qwen3-30B-A3B-Instruct-2507 on the Academic Cloud GPU cluster with a Python backend.**

I. Introduction

Picking the right cloud compute instance is not straightforward. Users need to check multiple provider pages, compare different pricing models, and deal with region-specific availability. The companion report covers data collection, schema normalization, and cross-provider cost analysis; this report focuses on the interactive platform that turns those insights into practical tools. The work builds on the CloudSpecs paper [1] and uses pricing data collected from AWS [2], Azure [3], and GCP [4] along with the Vantage aggregator [5]. The platform has three main features: (1) a *recommendation system* for filtering and ranking instances, (2) a *RAG chatbot* for answering cloud-pricing questions, and (3) a *shopping agent* that guides users through building infrastructure bundles.

II. System Architecture

The platform is a web app built with React that talks to an inference API on Academic Cloud [9] (`chat-ai.academiccloud.de`) serving two open-source LLMs. Instance data is loaded on the client side from JSON and CSV files derived from the merged multi-cloud dataset. Table I lists the main pages of the application.

III. Data Pipeline and Indexing

We collected pricing data from three major cloud providers: AWS [2] (1 121 instances via Vantage [5]), Azure [3] (1 120 instances), and GCP [4] (287 compute instances plus a 30.16 MB billing export that also covers GPUs, VMware Engine, and AI accelerators). The seven files we collected are:

1) `aws_all.csv` — 210.67 KB, all EC2 specifications.

TABLE I
Application pages

Page	Purpose
Top Instances	Weighted recommender across providers
AI Assistant	RAG chatbot with dual-prompt strategy
Shopping Agent	Guided infrastructure bundle builder
AWS / Multi Cloud	Provider-specific & cross-provider charts
About / References	Project info and data sources

2) `all_clouds_3sets_merged.csv` — 391 KB, the schema-normalized merged dataset.
3) `all_clouds_merged.json` — 781 KB, JSON variant used by the frontend at runtime.
4) `Pricing for My Billing Account.csv` — 30.16 MB, GCP billing export.
5) `aws_benchmark.csv` — 6.25 KB, CoreMark and FFmpeg benchmark scores.
6) `cloud_pricing_weekly.md` — 8 MB, weekly pricing snapshots.
7) `vantage_cloud_instances.md` — 28.81 KB, instance metadata from Vantage.

All seven files were uploaded to the **Arcana** vector-database service [9]. After indexing with `qwen3-embedding-4b`, the optimized vector store reached 107.37 MB across 3 947 chunks, which the LLM can quickly search through when a user asks a question.

IV. Multi-Cloud Recommendation Engine

The recommendation page gives users a hands-on way to explore cloud instances across all three providers—AWS, Azure, and GCP—in one place. Users can set filters like minimum vCPUs, minimum RAM, maximum hourly price, operating system, and whether they want spot or on-demand pricing. As soon as a filter is changed, the list updates and starts to shorten, showing only the instances that match.

Behind the scenes, the system scores each matching instance based on how much compute power and memory it offers per dollar. The top 15 results are shown in a ranked table so users can quickly see which instances give the best value for their

budget. Since benchmark data (CoreMark) is only available for AWS, the ranking stays fair across providers by default.

The idea is simple: instead of opening three different provider websites and manually comparing specs, users get a single view where they can filter, compare, and find the best option in seconds.

V. RAG-Powered AI Assistant

A. Models

We use two open-source LLMs, both running on the Academic Cloud GPU cluster with no fine-tuning: **Meta-Llama-3.1-8B-Instruct** [7] handles the RAG-based answers—it gets the user's question along with relevant data chunks retrieved from Arcana. **Qwen3-30B-A3B-Instruct-2507** [8] is a larger 30 B parameter model with broad world knowledge; we use it for rewriting prompts (explained below) and for answering general cloud questions that don't need RAG context. At query time, Arcana automatically prepends the top-3 most relevant chunks to the LLM's context.

B. Dual-Prompt Strategy

A big challenge is that users often ask vague questions like *"I want to build a YouTube app"*—this doesn't match well with the structured pricing data we have. To handle this, we set up a two-stage prompting pipeline:

Stage 1 — Prompt Rewriting. The raw query is sent to the Qwen model with a system prompt:

> *"You are a prompt rewriter. Rewrite the user input into an instance-focused query optimized for retrieval from the RAG documents. Prefer phrasing around specific instance types, sizes, families, regions, and price caps. Output ONLY the rewritten prompt text."*

So a vague question like the YouTube example gets turned into something like: *"Compute instances with 4–8 vCPUs, up to $0.50/hr, in us-east-1, suitable for video transcoding."*

Stage 2 — RAG Generation. This rewritten query then goes to the Llama model along with the Arcana context. We tell the model to always respond in strict JSON with keys like `summary`, `followup_question`, and `cards` (each card has `title`, `provider`, `price_hour`, `specs`, etc.). The frontend parses this JSON and renders it as instance cards instead of a plain chat thread—so the output looks clean and provider-centric rather than like a typical AI chat (Figure 1).

VI. Conversational Shopping Agent

The Shopping Agent takes a different approach from the chatbot—instead of answering a single question, it walks the user through building a complete infrastructure bundle step by step. The idea is that a user might not know exactly what they need, so the agent asks questions and narrows things down.

The user starts by describing what they need in natural language. The agent uses NLP to understand the requirements and, if anything is unclear, asks follow-up questions. Once the requirements are set, it shows related compute instances, database options, and storage options—and then wraps everything up with a summary showing itemized hourly and monthly costs (Figure 2).

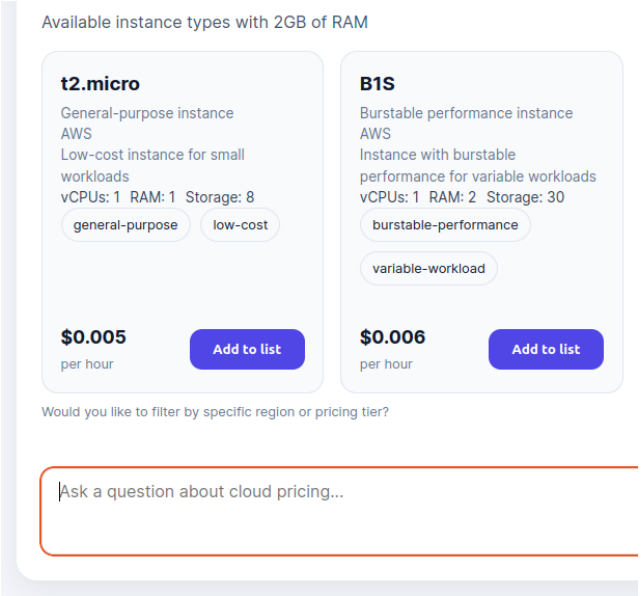

Fig. 1. AI Assistant: natural-language query → rewritten prompt → RAG retrieval → structured instance cards.

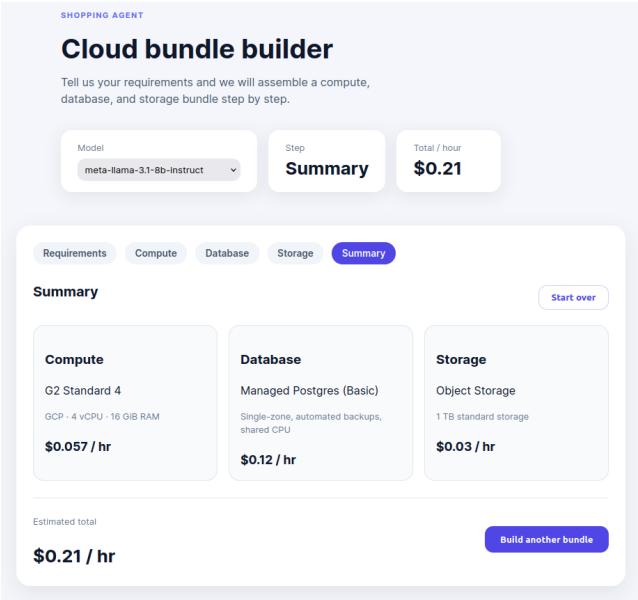

Fig. 2. Shopping Agent: step-by-step bundle builder with itemized pricing summary.

VII. Data Freshness and Implementation

Cloud pricing changes frequently, so we planned a weekly refresh mechanism. A script fetches the latest data from the Vantage API [5] (`instances.vantage.sh`)—which was the reference source from the CloudSpecs paper [1]—re-uploads the files to Arcana, and triggers re-indexing. This way the RAG assistant always answers based on fresh pricing instead of stale snapshots. Vantage also provides an MCP (Model Context Protocol) server, which opens the door for direct live tool-use integration with the LLMs in the future.

On the implementation side, the frontend is built with React 18, TypeScript, and Vite, with React Router v6 handling navigation. The LLM backend exposes an OpenAI-compatible `/v1/chat/completions` endpoint. Arcana uses `qwen3-embedding-4b` embeddings (DB version 7, CLI 0.3.5) and returns the top-3 relevant chunks per query, each with a relevance score and reference annotations for traceability. Infracost [6] was also evaluated as a complementary cost-estimation tool during the research phase. Runtime data is stored client-side in JSON, derived from the merged CSV via a sync script.

VIII. Conclusion

We built a cloud advisory platform that brings together a recommendation system, a RAG chatbot with dual-prompt rewriting, and a conversational shopping agent—all running on open-source LLMs hosted on academic infrastructure. The platform lets users compare instances across AWS, Azure, and GCP in one place, ask natural-language questions about pricing, and build complete infrastructure bundles through a guided conversation. Future work includes integrating the Vantage MCP server for live tool-use and extending the shopping agent to cover networking and container services.

References

[1] A. Iosup *et al.*, "CloudSpecs: Cloud Hardware Evolution Through the Looking Glass," 2023. https://www.cs.cit.tum.de/fileadmin/w00cfj/dis/papers/cloudspecs-final.pdf

[2] Amazon Web Services, "AWS Pricing," https://aws.amazon.com/pricing/, accessed Feb. 2026.

[3] Microsoft Azure, "Azure Pricing," https://azure.microsoft.com/en-us/pricing/, accessed Feb. 2026.

[4] Google Cloud, "GCP Pricing," https://cloud.google.com/pricing, accessed Feb. 2026.

[5] Vantage, "Cloud Instance Pricing," https://vantage.sh/, accessed Feb. 2026.

[6] Infracost, "Cloud Cost Estimates for Terraform," https://www.infracost.io/, accessed Feb. 2026.

[7] Meta AI, "Llama 3.1 Model Card," 2024. https://ai.meta.com/llama/

[8] Alibaba Cloud, "Qwen3 Technical Report," 2025. https://qwenlm.github.io/blog/qwen3/

[9] Academic Cloud, "Arcana RAG Service Documentation," https://chat-ai.academiccloud.de, 2026.

[10] M. Buffaloğlu, "CloudSpecs Blog Post," 2026. https://muratbuffalo.blogspot.com/2026/01/cloudspecs-cloud-hardware-evolution.html